

REVIEW ARTICLE

Scalability and Limitations of Existing Software Requirements Prioritization Techniques: A Systematic Literature Review

Muhammad Yaseen¹ | Waqar Mehmood¹ | Fazal Hameed² | Muhammad Asif Nauman¹

¹Faculty of Computing, Riphah International University, Lahore, Pakistan | ²Department of Computer Science and Engineering, American University of Sharjah, Sharjah, UAE

Correspondence: Muhammad Yaseen (muhammad.yaseen@riphah.edu.pk)

Received: 2 November 2024 | **Revised:** 20 May 2025 | **Accepted:** 16 July 2025

Keywords: functional requirements | nonfunctional requirements | requirement engineering | requirement prioritization

ABSTRACT

Requirements prioritization puts more emphasis on the software requirements based on their importance, making it a crucial activity in managing software requirements throughout the software development process. This work explains the concept of prioritization techniques, their relevance, and related issues. Recent literature has described many requirement prioritization techniques, but each comes with its own limitations and challenges, specifically regarding scalability. In this article, we provide a systematic literature review (SLR) of the requirement prioritization techniques over the past decade and identify their limitations and scalability issues. In this context, we develop a planned protocol that includes all the necessary steps required for the SLR process. As a result of conducting the SLR, 53 primary studies were shortlisted for data extraction. The analysis of the results indicates that there is a minimal focus on large-scale functional requirements, with only 9% of the work addressing this area. To the best of our knowledge, no work has been done on prioritizing requirements between ERP systems and mobile-specific applications. Most existing studies focus on imaginary or simulated projects, and very limited actual work has been carried out during the execution of the industrial projects. When requirements are prioritized correctly, the likelihood of successful implementation will be high, and it will lead to a higher quality product.

1 | Introduction

Requirement engineering (RE) can be defined as a software engineering (SE) process that focuses on understanding how the software requirements can be acquired systematically and appropriately from or on behalf of the users [1]. There are four types of requirements: business requirements (BRs), user requirements (URs), system requirements (SRs), and process requirements (PRs). BRs focus on the goals and objectives of the system, including profit and benefits delivered by the system to the clients or the business. URs represent the user and the organization requirements that need to be incorporated in the development of the product. SRs are typically subdivided into two main categories, namely, functional requirements (FRs)

and nonfunctional requirements (NFRs). FRs are those requirements that are basic or must-have requirements for the software system, ranging from a few to potentially thousands [2]. On the contrary, NFRs refer to the quality characteristics of the software that impact the performance, dependability, security, productivity, precision, accessibility, transferability, and compatibility of the software systems [3]. PRs cover a wide range of criteria such as cost control, development cost, time, customer satisfaction, competitive advantage, time for development, marketing style, quality of the constructed system, rate of production, and compliance with standards [4].

Software requirements prioritization (RP) gives significance or importance to one requirement over another; assigning

appropriate priority to requirements is essential for the successful implementation of a project. The prioritization of requirements is an essential process for SE where different criteria, such as business value, cost, and stakeholder preferences, determine the ordering of software requirements. A proper requirement prioritization (RP) process gives priority to fundamental needs first, which minimizes resource waste and enhances software quality [5]. Practitioners have advocated the employment of several techniques, including pairwise comparison technique, cost value approaches, and machine learning methods. The efficacy of these techniques highly depends on the project's scale and the degree of its measure [6]. Once the URs are collected and SRs are listed, a process called requirement management (RM) is initiated. RM is a systematic and structured approach of recording, planning, evaluating the requirements, managing changes to the requirements, and communicating those changes back to the stakeholders. Although RM is an ongoing process throughout a project, it becomes especially critical during the final phase of software design and development. The priority has played a significant role in requirements management as software development has become more sophisticated in recent years [7]. Often, when it comes to implementing a mammoth software solution like the ERP system, the requirements are equally vast. When the stakeholder aims to meet all the requirements that are set within the available time and funds, it becomes imperative to prioritize these requirements [8, 9]. This process of prioritizing the requirements is particularly important because of the fact that all the types of requirements are interrelated to each other. This RP becomes even more critical in the scenario of a complex system where many stakeholders and users have differences in perceptions of the requirement priorities [8].

Scalability in RP refers to the ability of employing prioritization techniques to a wide range of software requirements as a project grows in size without compromising its performance. This issue becomes particularly important in the context of software projects that have large numbers of requirements exceeding thousands in many cases and needs to be properly captured and prioritized [10]. Traditional ranking methods, such as AHP and numerical assignment, face scalability challenges because of performance issues and computing constraints. To address these limitations, machine learning and automatic methods have been employed that not only improve quality but also provide faster processing in large-scale prioritization processes [11]. To enable efficient incorporation of prioritization techniques in large-scale prioritization processes, several techniques have been developed that use both quantitative and qualitative approaches for prioritization. These hybrid methods of decision making combine operations research on multiple criteria with artificial intelligence. In addition, some of the prior works advocate the use of preprocessing techniques such as clustering and filtering to enhance the efficiency of RP. There have also been interesting developments in the automation of requirement classification using natural language processing (NLP) [12].

RP deals with the manner in which the requirements should be implemented. This prioritization typically stems from the stakeholder's perspective (URs) or the system perspectives (FRs and NFRs). The RP process is typically divided into three phases: (i) the planning phase in which the requirements are

arranged and refurbished for prioritization, (ii) the implementation phase in which actual prioritization occurs, and (iii) the documentation phase where the findings of the RP process are captured and disseminated [13]. There are two approaches to RP [13]: The first approach prioritizes the requirements based on the preferences of the stakeholders through negotiations, and the second technique prioritizes the requirements based on the decisions made by the software experts. On completion of the prioritization process, it is possible to compare both sets of requirements in terms of rank and importance [14].

Although a significant amount of work has been done in the RP, several challenges still exist. Many state-of-the-art techniques are suitable for BRs, PRs, and URs. However, none of these techniques effectively address the dependency issues of FRs while prioritizing PRs. Besides the importance, very little work has been carried out to prioritize the requirements of complex software systems like enterprise resource planning (ERP). This article presents an SLR process aimed at analyzing existing RP techniques, highlighting their advantages and limitations. To address the scalability issues of existing techniques, we need to identify how well these techniques scale in handling and prioritizing all types of requirements. Identifying these limitations will enable the researchers to fill the gap in the RP. The rest of the paper is structured as follows: Section 2 focuses on the literature review, Section 3 describes the research methodology, Section 4 provides the results, and Section 5 discusses the observations resulting from the SLR, followed by the conclusion and future work in Section 6.

2 | Background Study

The most prominent and widely used technique for RP is the analytical hierarchy process (AHP) technique, which performs pair-wise comparison between all requirements. This technique requires a total of $n(n-1)/2$ comparisons for n requirements and is used to prioritize every new requirement to its maximum level. For $n=10$, AHP will perform 45 comparisons of the requirements. As the number of requirements (i.e., n) increases, the time that it requires to process them increases drastically. In the case where the requirements size is in thousands, there will be a minimum of $1000*(1000-1)/2=499,500$ comparisons, which would take a lot of time, making it hard to implement. Therefore, the scalability issue of the AHP technique makes it unsuitable for complex systems with large requirements because of the pair-wise comparison for each requirement [15]. The framework in [16] categorizes the requirements as a function of benefit and cost that depicts the requirement interrelationships. This study introduces six approaches to handling dependencies. First, it assigns fixed values to costs and benefits for each requirement. Second, it groups requirements independently to handle complicated problems during actual calculations. Third, it requires the benefits to be tangible (e.g., dollars or time in hours). Fourth, it performs a pairwise comparison. Finally, it employs the use of discrete values (e.g., 1, 2, and 3) instead of continuous values. The main drawback of existing conventional prioritization methods is that they are unable to deal with the complexity, ambiguity, and inconsistency issues

[17]. This issue is addressed by adaptive fuzzy hierarchical cumulative voting (AFHCV) technique, which is based on cumulative voting. AFHCV blends adaptive mechanism with the existing fuzzy hierarchical cumulative voting (FHCV) scheme. FHCV addresses the ambiguity and complexity issue but does not give solutions on how to handle changes and prioritization of requirements during run-time. To deal with such problems, AFHCV comes into operation, which deals with how to prioritize run-time requirements or adjust prioritization values of the current requirements after the addition of new requirements. The analysis of results indicates that AFHCV is very accurate and gives results closer to that of actual priorities. The work done by [18] defined other prioritization methods like fuzzy logic and decision trees to create an intelligent procedure for ranking new emerging requirements by using parameters as inputs. This means that, depending on one scenario or another, within the framework, different parameters are determined. The author explained the binary tree during the RP using an algorithm. Requirements are first ordered, followed by the construction of a binary tree. After that, it employs a sorting method to enable prioritization either in ascending or descending order. This method is difficult to apply in practice. However, it is very useful because there are fewer comparisons compared with AHP. Thus, this technique transpires more effectively for projects that fall under the medium as well as large size of requirements [19]. However, the main issue with this technique is that the methodology only categorizes and sorts the requirements based on their priority but does not give a value to the requirements. Other studies on RP that utilize machine learning algorithms to prioritize a large set of FRs fail to incorporate dependency factors for all the FRs [20].

In addition to prioritizing FRs, the literature also rendered works on the priority of NFRs. In the work [21, 22], the author described three strategies for prioritization of NFRs. The first strategy treats the set of nonfunctional with the highest priority, as well as a set of FRs. The second strategy prioritizes the NFRs in the manner that they are considered more important than the FRs. The third approach involves prioritizing the NFRs as a separate list, though on a functional basis. It was evident from the experiments that the third approach is the most efficient, followed by the second approach, which is more effective than the first approach.

Previous research works [23–29] have performed systematic literature reviews (SLRs) on RP. However, none of them have identified prioritization methods that are suitable for specific types and sizes of requirements. In addition, these works have not discussed techniques that use both types and sizes of requirements using SL. Also, our research includes some more analysis factors that are not raised and discussed in the previous SLR.

3 | Research Methodology

The design framework for the proposed SLR is illustrated in Figure 1. As recommended by previous studies [7, 30–33], we have followed a structured process. This process begins with posing the research questions, followed by determining search

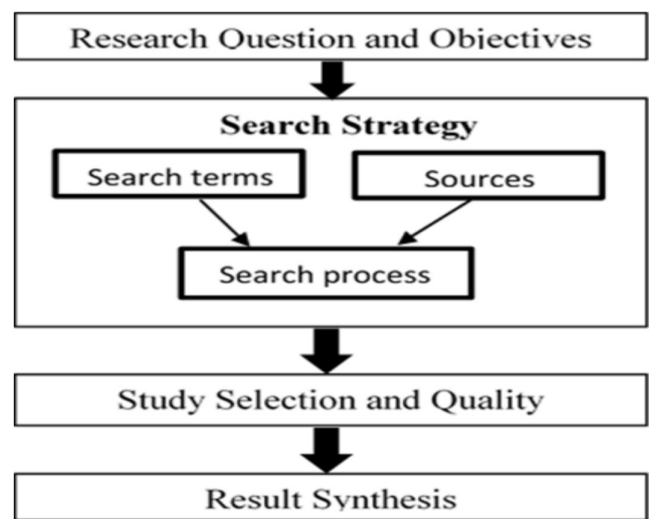


FIGURE 1 | Research design framework.

strategies, assessing and quantifying quality assessment criteria, and finally, integrating the collected data.

3.1 | Research Questions

It is worth mentioning that formulating the research questions is the first step of the research design process. These research questions guide the approach used in the research to produce appropriate answers. This is followed by establishing research objectives to answer the posed questions. In this research, the research questions are formulated as follows:

RQ1: What are the available techniques in the literature to prioritize software requirements of any type?

RQ2: What are the limitations of the current RP techniques, and to much extent the existing RP techniques are scalable to prioritize a large set of requirements?

3.2 | Search Strategy

The search strategy is a key step in conducting an SLR. In this study, the search strategy is divided into the following three steps based on the objectives and the formulated research questions.

3.2.1 | Identifying Search Terms

The search terms are obtained from the formulated research questions that help to search for a particular paper within various databases, such as digital libraries. Example search phases include “software requirements prioritization,” “prioritizing software requirements,” and “requirement prioritization in software.”

3.2.2 | Making of Search String

The following steps are followed for constructing a searching string for SLR:

- Major terms derivation: Major terms are derived based on the formulated research questions.
- The substitute spelling and synonyms of the major terms are identified from the literature.
- Boolean operators' connection of the major terms: The "OR" operator is applied in the case of synonyms to the major terms and the "AND" operator in the case of the combination of the major terms.

Result of Step 1: The following major terms are used:

1. Software requirements
2. Prioritization

Result of Step 2: Some synonyms for the major terms used in the research are provided below:

1. **Requirement engineering:** Elicitation OR analysis OR specification OR implementation of the requirement.
2. **Prioritization:** Ranking OR prioritization OR ordering.

Result of Step 3: The search string developed from Step 1 and Step 2 is as follows:

((Requirements engineering OR requirements elicitation OR requirements analysis OR requirements specification) AND (prioritization OR Ordering OR Ranking)).

3.2.3 | Digital Sources

Once the keywords were finalized, the formed search string queries were used in various digital libraries and indexes like IEEE Digital Library, Elsevier, Springer Link, ACM portal, and Google Scholar.

3.2.4 | Search Process

After performing a query in digital libraries, several publications related to RP were obtained. This process of searching for required papers was not an easy task to look for papers because the process involved inclusion and exclusion criteria. Table 1 provides the specifics of papers identified and included in the final sample following the application of inclusion and exclusion criteria, as well as the quality assessment criteria used (Table 1: selected publications).

3.3 | Study Selection and Quality Assessment

Specific inclusion and exclusion criteria must be employed when selecting the final set to be used as the main source of reference. The following flowchart in Figure 2 shows the SLR study selection process.

The criteria for selecting a particular study in SLR are based on Figure 2 and discussed as follows.

TABLE 1 | Papers identified from different digital sources.

Source	Total result	Primary selection (based on inclusion criteria)	Final selection (based on a quality assessment)
IEEE explore	245	30	19
ACM	22	8	0
Science Direct	173	23	10
Springer Link	180	27	12
Others	90	17	12
Total	710	105	53

3.3.1 | Inclusion Criteria

- The paper to be submitted should be written in the English language only.
- Papers associated with the prioritization of software requirements will be nominated.
- Papers that discuss a specific technique or application of a particular technique on precise data or applications will be selected.

3.3.2 | Exclusion criteria

- Studies related to RP but have no real-world application will be excluded.
- Survey papers, literature reviews, or SLR-based papers, as well as studies related to empirical studies, like interviews, will be excluded.
- More exclusion of papers will be based on quality assessment criteria.

To further refine the selection process, a set of quality assessment criteria is applied to the selected papers. This ensures that only high-quality papers are engaged as the main sources of information in this research work. Since quality is an abstract concept, this research quantified the quality based on some of the following factors, such as the quality of the journal or conference where the paper was published and whether the author of a particular paper has substantiated their method or not.

3.3.3 | Selecting Primary Sources

The prearranged selection procedure consists of two stages: The first stage involves a preliminary assortment from the papers of the search results, which could reasonably fulfill the criteria of selection. This stage starts with reading the heading and abstract of the papers; then, from a selected list of papers, it starts reading the abstract of every listed publication. Since

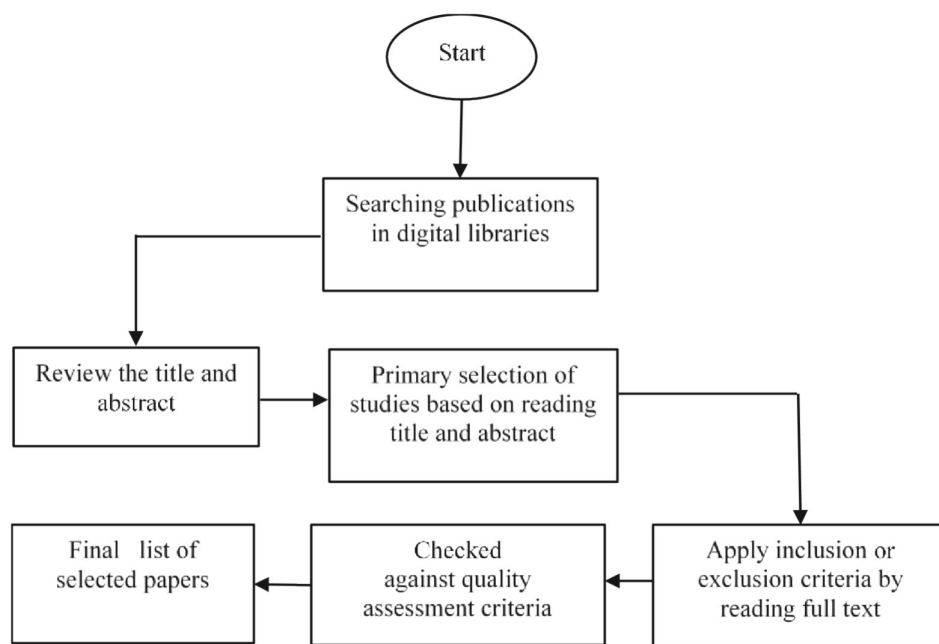


FIGURE 2 | Publication selection process.

the title or abstract is not enough because of insufficient details, the second stage starts with reading the whole paper from the selected list. An interrater reliability test was done to minimize the researcher's bias where the secondary reviewer selected five publications randomly from the list of “total results found” and did the initial and final selection processes. The results were matched with the outcomes made by the primary reviewer, and no differences were found. We have identified 54 papers, as shown in Table 1, where the matching papers have been removed from the finally selected list of papers.

3.3.4 | Publication Quality Assessment

We designed a quality assessment to guide the selection of the final papers. This assessment was performed in parallel during the extraction of the data. The checklist for quality consists of the following key questions:

- Is it clear how the selection process was done?
- Is it clear how the factors related to the limitations of prioritization techniques are identified?

The factors were either marked with a YES or NO response. Based on the results of this quality assessment, we finalized a list of 54 publications.

3.4 | Results Synthesis

The final step of the research design is data synthesis. Following the quality assessment, several key factors were identified as outlined below. These factors were finalized from previous SLR papers by different authors in their domains [7, 31]. This study gives importance to the size and the

type of requirements, as the research questions are based on them. The main reason is that a particular technique should be judged based on its suitability for either large size requirements or considering some specific type of requirements. The size of requirements is based on the number of requirements in software. Sizes of requirements are broadly divided into three categories, i.e., small, medium, and large. From literature, small size is supposed to have few requirements of a maximum size of 10–12, where pairwise comparisons become more feasible. Although large size is in the hundreds, anything in between is classified as medium size. The type of requirements refers to how specifically the technique is designed for a particular type of requirement [34].

- Time period
- Type of publication (journal/conference)
- Research methodologies
- Technique applied on actual software (yes)
- Type of author (student/industry/other)
- Techniques applied (algorithmic/nonalgorithmic)
- Type of software (mobile/web/business or university software/entertainment)
- Size of requirements
- Type of requirements

4 | Results

To achieve certain goals and to answer specific research questions, first, we have to identify all possible techniques from the literature and analyze them. The following subsections describe

all the techniques, along with how scalable they are and what the limitations of existing techniques are.

4.1 | Techniques Identified as the Result of SLR

To address RQ1, the following Table 2 shows the techniques identified with a brief description as a result of SLR.

4.2 | Factors Analysis

Because of space limitation, we used numeric values for every factor value as mentioned in Section 3.4. The following numerical values are assigned for each value:

- **Time period:** 1.00 = “below 2000,” 2.00 = “2000–2010,” 3.00 = “2010–2015,” 4.00 = “after 2015”
- **Publication type:** 0.00 = “conference,” 1.00 = “journal”
- **Research methodology:** 1.00 = “mathematical modelling,” 2.00 = “experiments,” 3.00 = “case study,” 4.00 = “other,” 5.00 = “case study + experiments”
- **Techniques applied on real software:** 0.00 = “no,” 1.00 = “yes”
- **Author type:** 1.00 = “student,” 2.00 = “professor,” 3.00 = “industry,” 4.00 = “other”
- **Technique applied:** 1.00 = “algorithmic,” 2.00 = “nonalgorithmic”
- **Software type:** 0.00 = “not available,” 1.00 = “mobile,” 2.00 = “website,” 3.00 = “business software”
- **Size of requirements:** 0.00 = “small,” 1.00 = “medium,” 2.00 = “large,” 3.00 = “very large”
- **Type of requirements:** 1.00 = “process,” 2.00 = “user,” 3.00 = “functional,” 4.00 = “nonfunctional,” 5.00 = “process and user,” 6.00 = “process and nonfunctional,” 7.00 = “functional and nonfunctional,” 8.00 = “not”

Table 3 shows factor analysis based on all the quantified factors against each of the research articles.

4.2.1 | Time Period

The number of papers retrieved from different sources is published in different decades. This factor shows how much work has been carried out within a particular period of time and up to what extent limitations are addressed. Figure 3 shows a number of publications found in different time periods. The details are given in Table 4.

The results showed that 30.2% of work has been carried out in the last 3 years, which indicates the importance of work on RP at the current time. This is due to a change in organizational culture where many organizations are adopting software

in line with the advances in web technologies and software outsourcing.

4.2.2 | Type of Publication (Journal/Conference)

The survey revealed that out of 53, 52.8% (28) publications belong to journals, whereas 47.2% (25) are conference papers (including one workshop paper). Out of the 47.2%, more than 90% of the publications are IEEE proceedings, whereas in the journal category, more than 90% of the publications belong to Springer and Elsevier, which are retrieved, indicating the quality of the publications. Figure 4 shows the breakdown between the journals and conference papers in the final set of publications.

4.2.3 | Research Methodologies

In analyzing the research methodologies, the literature survey showed a high percentage of 43.4% adopted an experimental approach using real software requirement data, 34% adopted case studies as a research method, and 5.7% of researchers used a combination of experiments along with case studies. Only 32% of the work used pure mathematical modeling based on a matrix or other criteria but without any experiments on real software requirement data. Case studies and experimental evaluations were either based on mathematics or algorithmic solutions. The technique and publication are considered the best in cases where experiments are conducted, especially on real software. As the majority of papers belong to highly reputable journals, they used either experiments or case studies as tools for the evaluation of a particular technique.

Figure 5 and Table 5 show the details of research methodologies applied across different publications in requirements sys.

4.2.4 | Actual Software (Yes/No)

Next, the survey examined whether the evaluation of techniques was applied to real software or only simulation. Figure 6 shows the percentage of authors who have applied a particular technique to the software system.

From the SLR results, 56.6% of authors applied their technique to real software. However, all techniques were validated during the experiments.

4.2.5 | Type of Author (Student/Industry/Other)

Analysis of the authors revealed that the majority of studies were conducted by students (75.5%). Only a very small percentage (3.8%) of the research was contributed by the industry. This indicates the lack of scholarly research being carried out on RP in the industry, hence bringing the question of whether the industries are aware of the options to prioritize the software requirements to increase the rate of success in a software development project. Figure 7 shows the detailed studies conducted by the authors.

TABLE 2 | Techniques identified as a result of SLR.

Technique	Description
AHP [2, 10, 15, 30–33, 35–37]	This technique compares one requirement with another in the generation of a comparison matrix, which aids in the determination of the priority of each requirement.
Attribute goal-oriented [31, 38]	In the case of the goals of the stakeholder and the contributed value of each requirement, graphical representation and analysis are done.
Benefit and cost [39, 40]	This technique involves the comparison of the value of each requirement of the organization with the cost and then draws a ratio of prioritization.
Binary search tree [9, 41]	In this technique, all requirements are ordered like parent and child nodes in a tree structure.
Binary tree [14, 18, 39]	Since comparing each requirement against each other is quite tiresome, this technique sorts requirements into a tree and then generates an algorithm that decreases the number of comparisons in nodes and then provides a weightage score to each requirement and prioritizes them.
Binary priority list [42]	This technique prioritizes requirements based on how it is important and how much benefit they will have on the application domain.
Case-based ranking [33]	This technique employs a machine learning procedure for reducing the time and effort necessary during requirement elicitation for prioritizing requirements according to stakeholders' preferences.
CBPA [4]	This technique prioritizes requirements where interrelationship matrices depict the relationship between them from several viewpoints.
Cost value ranking [3, 43]	This technique prioritizes requirements by deducing the relative cost of every requirement by pairwise comparisons through AHP.
Cognitive-driven requirement prioritization [43]	This technique is a combination of analytical hierarchy process self-organizing maps and cognitive psychology to construct a conceptual model that is capable of accessing stakeholder knowledge and preference to filter requirements.
Cumulative voting [44–46]	This method involves identifying a particular amount of votes, which is then divided on each of the requirements. The requirement of getting a high number of votes has much priority.
Hierarchy AHP [47]	In this technique, first, requirements are put in order and then rated using the AHP technique. It is the improved AHP that reduces the number of comparisons.
Interactive GA [48]	A method is proposed to introduce and rank system requirements with the help of an algorithm in the form of graphs.
Lanchester theory [49]	This technique utilizes Lanchester theory with the aid of analyzing customer profile needs to sort out requirements based on the market requirements.
Numerical assignment [45, 50, 51]	The numerical assignment involves sorting out requirements into high-, medium-, and low-priority levels.
Value oriented (goal based) [52, 53]	Requirements are related to its business value and prioritized depending on its value in the eyes of the stakeholders.

(Continues)

TABLE 2 | (Continued)

Technique	Description
Value based [54, 55]	To be specific, sometimes a specific need is crucial for one enterprise, but at the same time, that need is irrelevant for the other enterprise. Hence, there is a need of a framework that will be able to give priority to requirements based on their importance.
Value-based intelligent requirement prioritization [56–58]	This method employs the use of experts, stakeholders, and fuzzy logic to rank requirements.
Group recommended algorithms [59–61]	This technique follows the decision of a group on the relative importance of requirements
Hierarchical cumulative voting [12, 62, 63]	This approach is the fusion of AHP and CV in which the requirements are positioned hierarchically, and then, the points are distributed among the requirements in each group individually.
Machine learning [16, 17, 64]	However, some techniques utilize some of the machine learning techniques like clustering to enhance software requirements prioritization.
QFD [65, 66]	Quality functional deployment is used to make priorities based on requirements in the market.
Situation transition based [67]	This technique involves factors such as what people need, and want, the state of the environment, and the technical problems in that specific context to rank needs.
ANN [8, 68]	Training sets and artificial neural networks are used to learn and determine the priority of requirements.
Contextual preference-based technique [69–71]	A method developed for achieving more nonfunctional characteristics in various circumstances such as user expectations, designer problems, and other technical concerns.
DRANK algorithm [72]	An algorithm is proposed for ranking requirements based on the preferences of the stakeholders and relative dependencies among requirements.
Fuzzy decision logic [13]	It is possible to generate decision rules utilizing the theory of fuzzy logic and illustrate a model for prioritizing requirements under these circumstances
SNIPR [73–75]	A method based on natural language processing is employed to prioritize requirements.
Tensor [76–78]	In this technique, they are all included in one tensor, then decompose them, and later prioritize them breaking the functional requirements, the nonfunctional requirements, and the stakeholder's preferences.

4.2.6 | Techniques Applied (Algorithmic/Nonalgorithmic)

In the previous work, the software RP techniques applied are mainly either algorithmic or nonalgorithmic. Nonalgorithmic techniques are based on either mathematics, tools related, or depend on judgments. Figure 8 shows the detailed distribution of different techniques applied.

The figure showed that 66% of the studies applied nonalgorithmic techniques, whereas 34% applied algorithmic techniques. Algorithmic techniques are best for prioritization of medium- to big-sized requirements, but they are difficult to use compared

with nonalgorithmic techniques, which face scalability and limitation issues.

4.2.7 | Type of Software on Which the Techniques Are Applied

Different types of software were used while applying different RP techniques. Figure 9 shows the detailed distribution of the software types. Based on the figure, very few techniques were applied on mobile platforms. This indicates a promising focus on developing new software RP techniques.

TABLE 3 | Factors identified from different research papers.

Paper ID	Time period	Publication type	Research method	Technique applied on real software	Author type	Technique applied	Software type	Size of requirements	Type of requirements
1	4	1	4	0	4	1	0	0	7
2	2	1	3	1	3	2	3	0	5
3	2	0	1	0	1	2	0	0	6
4	2	0	2	0	1	1	0	1	1
5	3	1	3	1	3	1	3	0	2
6	0	0	3	1	1	2	0	0	1
7	2	0	1	1	1	2	3	0	5
8	2	0	2	1	1	2	0	0	4
9	2	0	2	0	1	2	0	0	5
10	3	1	2	1	1	1	0	1	5
11	2	0	3	0	1	2	0	0	8
12	2	0	3	1	1	2	3	1	3
13	3	0	2	0	1	3	0	0	8
14	3	0	1	0	4	1	0	0	2
15	3	0	2	0	1	2	0	0	7
16	2	0	2	0	1	2	0	0	5
16	2	0	2	0	1	2	0	0	5
17	4	0	2	0	1	1	0	0	4
18	2	0	2	0	1	1	0	1	8
19	3	0	5	0	1	2	0	0	8
20	4	0	3	1	1	1	2	2	2
21	3	1	2	1	4	1	3	1	2
22	4	0	2	0	1	1	0	0	8
23	4	0	2	0	1	2	0	0	8
24	3	1	2	1	4	1	3	0	4
25	4	1	2	1	1	2	0	0	3
26	4	1	3	1	1	2	1	1	3

(Continues)

TABLE 3 | (Continued)

Paper ID	Time period	Publication type	Research method	Technique applied on real software	Author type	Technique applied	Software type	Size of requirements	Type of requirements
27	4	0	3	1	4	2	3	0	2
28	2	1	3	1	1	2	2	1	2
29	2	1	5	1	4	1	3	3	2
30	3	1	2	1	1	1	2	2	5
31	4	0	3	1	1	2	3	1	2
32	3	0	3	0	4	2	0	0	8
33	2	1	3	0	1	2	0	0	6
34	4	1	2	0	1	2	0	0	2
35	4	1	2	0	1	1	3	1	5
36	4	1	3	1	1	1	3	1	6
37	4	1	2	1	1	2	3	2	3
38	4	1	2	1	1	2	3	0	3
39	2	1	3	1	1	1	3	1	3
40	4	1	2	1	1	2	3	0	7
41	3	1	2	1	2	2	0	1	3
42	4	1	4	1	1	2	0	0	4
43	3	1	3	1	1	2	2	0	3
44	2	1	5	1	1	1	4	0	3
45	3	1	2	1	4	2	3	0	4
46	3	0	1	0	1	2	0	0	4
47	3	1	1	0	1	2	0	0	4
48	3	1	2	1	1	2	4	0	3
49	2	1	3	0	4	2	3	0	1
50	1	0	3	1	1	2	3	2	3
51	1	0	3	1	1	2	3	0	5
52	2	0	1	0	4	3	0	0	1
53	2	1	1	0	1	2	0	0	4

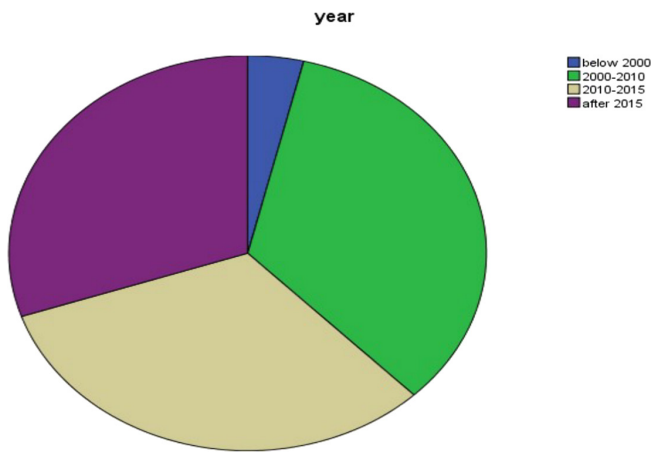


FIGURE 3 | Time period distribution.

TABLE 4 | Frequencies and percentages of time periods.

Time period	Frequency	Percentage (%)
Below 2000	2	3.8
2000–2010	18	34
2011–2015	17	32.1
2015 onwards	16	30.2
Total	53	100

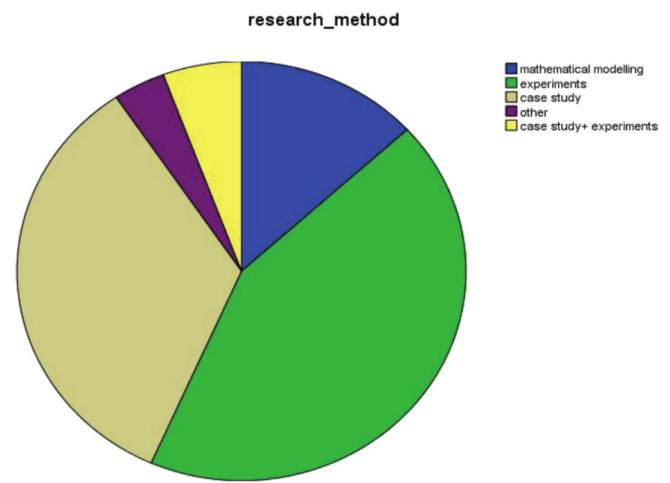


FIGURE 5 | Distribution of research methodologies.

TABLE 5 | Frequencies and percentages of different research methodologies.

Research methods	Frequency	Percentage (%)
Mathematical modelling	3	7
Experiments	21	48.8
Case study	15	34.9
Other	2	4.7
Case study + Experiments	2	4.7
Other	43	100

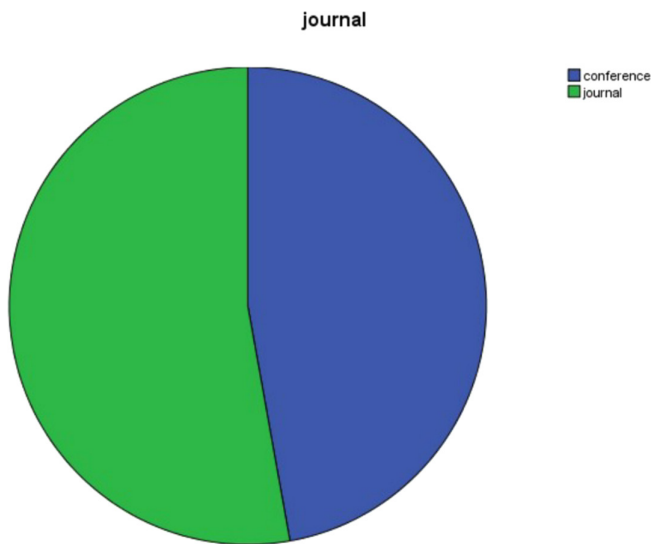


FIGURE 4 | Distribution of conferences and journals.

4.2.8 | Size of Requirements

Different techniques were applied based on different sizes of requirements, as shown in Figure 10. As illustrated, 68% of the requirements on which the new techniques were tested were of small size. 22.6% of requirements were medium-sized, and only 9.4% of techniques used large requirements as input. Results show that only 9% of techniques are suggested and applied for

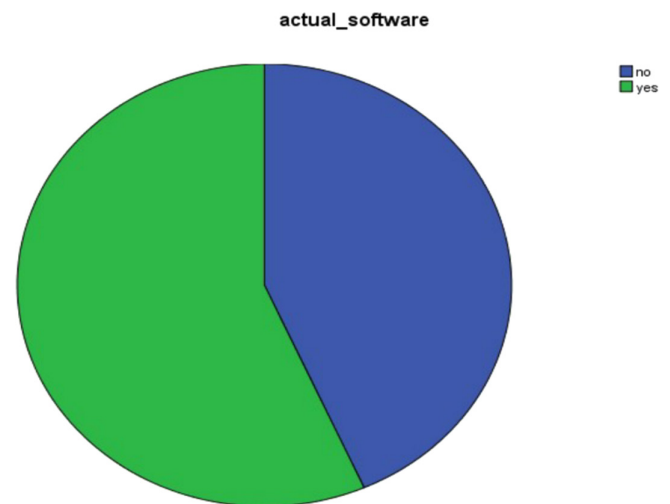


FIGURE 6 | Distribution of the presence of actual software.

large-sized requirements only. Nonetheless, no studies considered requirements input from an ERP system where the features may reach into thousands. This indicates a clear gap in proposing a new RP technique that can cater to FRs of the ERP systems.

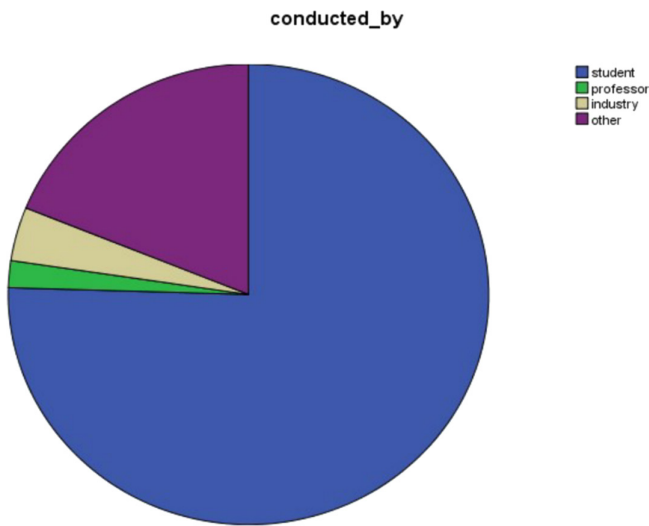


FIGURE 7 | Distribution of studies conducted by researchers.

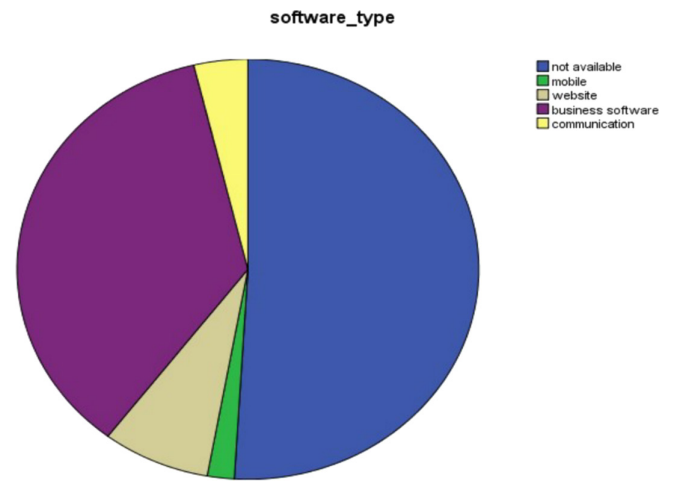


FIGURE 9 | Distribution of software type.

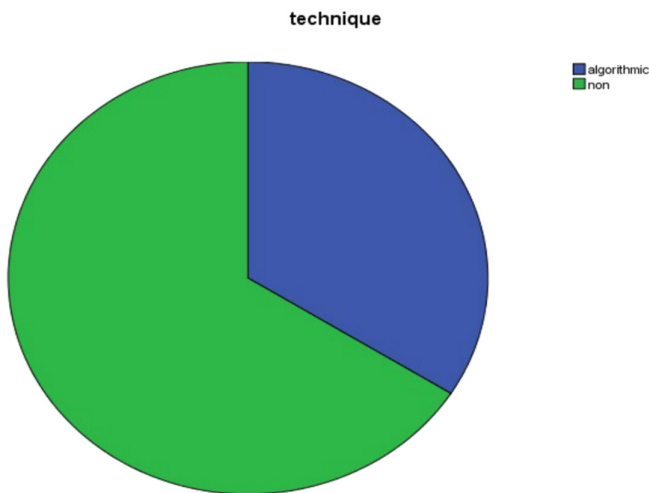


FIGURE 8 | Distribution of different techniques applied.

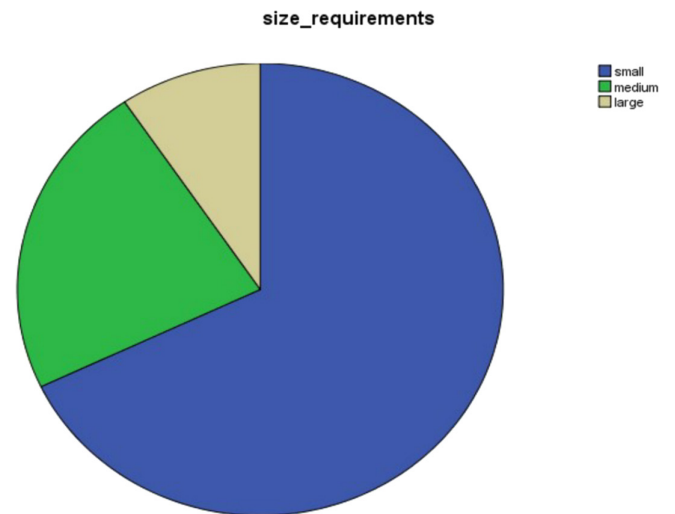


FIGURE 10 | Distribution of different size requirements.

4.2.9 | Type of Requirements

While instituting the RP techniques, various types of requirements were addressed. Table 6 and Figure 11 highlight various types of requirements that are considered for prioritization experiments based on the literature. PR and NFR have fewer techniques in comparison with FR. It is evident that more techniques are used for FRs than for PR and NFR.

4.2.10 | Size of the Software Requirements Against Types of Requirements

Finally, an analysis was carried out on the types of requirements against the size of requirements reported in the RP literature. Table 7 and Figure 12 show the detailed analysis of the comparison.

As shown in Table 7, it is concluded that the majority of the proposed RP techniques were applied to small projects (68%), and only a small percentage of the techniques (9.4%) were applied to

large-scale projects. This motivates the study to investigate the performance of RP techniques on large software development project requirements.

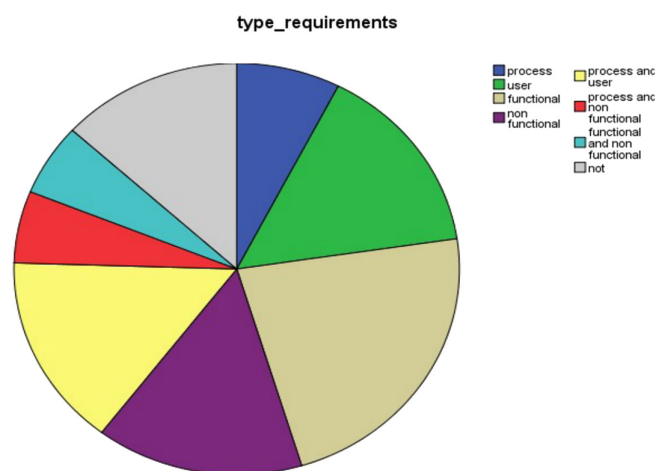
4.2.11 | Limitations of the Existing Techniques

In analyzing the techniques in detail to address RQ2, we have taken certain variables from Section 4.2, as shown in Table 8, followed by the analysis that is summarized in Table 9.

According to Table 9, different techniques are recommended for prioritizing different types and sizes of requirements. These techniques are the result of modification or improvement to overcome the limitations of existing techniques under different conditions and circumstances. For instance, AHP is a simple and widely applied technique for prioritizing a small set of requirements. However, AHP fails on big projects with a large set of requirements. AHP is better for small-sized projects having less than 10 requirements of any type. For prioritizing BRs, which are less in magnitude, techniques such as “AHP,” “benefit

TABLE 6 | Percentages of different types of requirements.

Type of requirements	Frequency	Percentage (%)
Process requirement (PR)	4	7.5
User requirement (UR)	8	15.1
Functional requirement (FR)	12	22.6
Nonfunctional requirement (NFR)	8	15.1
Process and user requirement (PR-UR)	8	15.1
Process and nonfunctional requirements (PR-NFR)	3	5.7
Functional and nonfunctional requirements (FR-NFR)	3	5.7
Not mentioned	7	13.2
Total	53	100

**FIGURE 11** | Distribution of different types of requirements.

and cost,” “voting,” “value-based,” “value-oriented,” and “group recommendation” have been successfully applied, and these techniques are well-suited for business-relevant requirements. For NFRs, some researchers have suggested techniques like “contextual preference-based technique” and “QFD.” However, techniques like “AHP,” “case-based ranking,” “CBPA,” and “Tensor” methods are also suitable for prioritizing NFRs. We note that all suitable techniques designed for BRs and NFRs work well in both categories, as both kinds of requirements are often small in size.

Regarding URs, most techniques listed in Table 9 are presented, but they are well-suited for the scenario when the number of requirements is less in size, such as “AHP,” “majority voting,” and “Interactive GA.” However, these techniques do not work well because they take a long time to calculate the priority of large requirements. As comparing requirements against others is time-consuming, the binary tree technique was proposed to arrange requirements in a tree and generate the algorithm. The tree-based approach reduces the number of comparisons in nodes and assigns a weightage score to each requirement for prioritization. In this technique, requirements are structured in the form of a tree. There is no specific rule based on which requirements can be prioritized, but it is a totally user-dependent process. Users will decide which requirement has more priority as compared with others. Interdependency of requirements with one another is still a big issue in this technique. This technique does not automatically assign priority values to requirements, but stakeholders must assign them [18]. For the prioritization of a large set of requirements, machine learning techniques have been proposed. For instance, the research in [16] used the K-mean clustering technique to assign priority to a requirement based on the importance of the particular requirement. The case study was applied to a website data of seven airports, where priority is assigned based on services in terms of passengers at the airport. The study demonstrated the approach by categorizing services into four clusters representing the level of importance, i.e., very important, important, less important, and not necessary. Although priority was given based on relative importance, other information is still missing about how FRs need to be prioritized before development so that key requirements could be assigned high priority as compared to others. Similarly, other

TABLE 7 | Percentages of different sizes of requirements against types of requirements.

Types of requirements	Small (%)			Medium (%)			Large (%)		Total (%)
PR or BR	3	5.6	1	1.8	0	0	4	7.5	
UR	4	7.5	3	13.2	1	1.8	8	15	
FRs	5	9.4	4	7.5	3	5.6	12	22.6	
NFRs	8	15	0	0	0	0	8	15	
PR and UR	5	9.4	2	3.8	1	1.8	8	15	
PR and NFRs	2	3.8	1	1.8	0	0	3	5.6	
FRs and NFRs	3	5.6	0	0	0	0	3	5.6	
Not mentioned	6	11.3	1	1.8	0	0	7	13.2	
Total	36	68	12	22.6	5	9.4%	53	100	

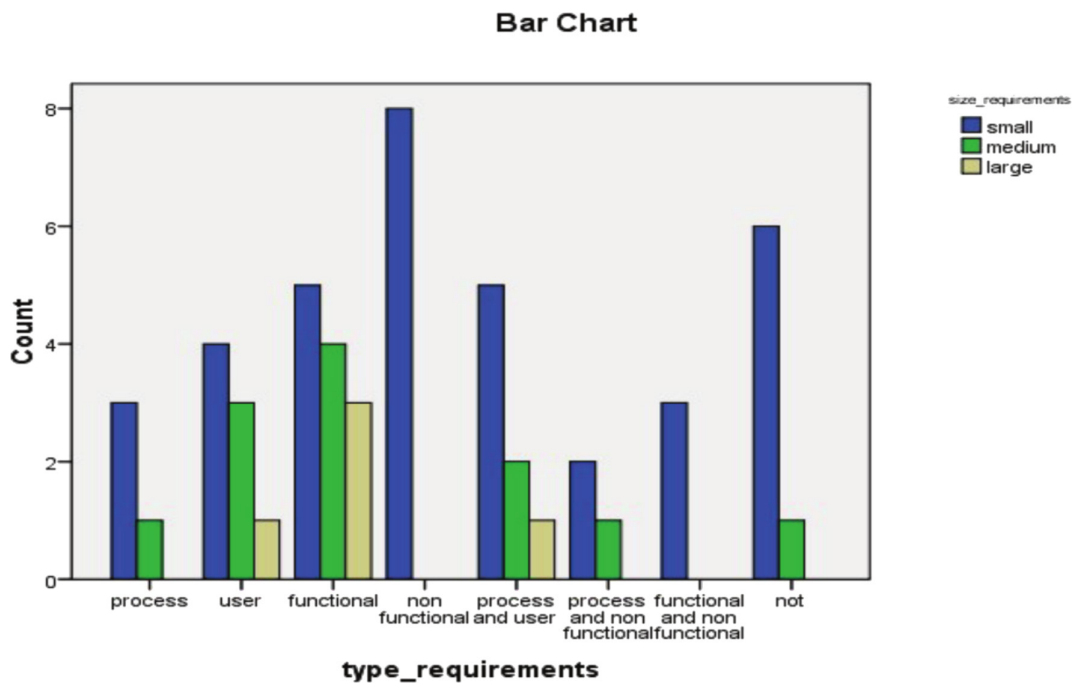


FIGURE 12 | Distribution of types of requirements against the size of requirements.

TABLE 8 | Variables for analysis of techniques.

Variables	Description
Usability	Some techniques are simple to use and prioritize requirements, whereas some are difficult and complex.
Algorithmic vs. nonalgorithmic	Some techniques are algorithmic, whereas others are nonalgorithmic.
Time-consuming techniques (Yes/No)	Some techniques take time in comparisons and prioritizing requirements, whereas others do not.
Suitable size of requirements	Some techniques are suitable for prioritizing small-sized requirements, whereas others are for large sets of requirements.
Suitable for type of requirements	Some techniques are designed for prioritizing functional requirements, some are for nonfunctional, whereas some are for business requirements. Some techniques can be used for a combination of any two or three.

works that applied clustering techniques for prioritizing large requirements failed to raise dependency issues in FRs. For instance, one paper presented a machine learning approach for requirements during elicitation to reduce the efforts during prioritization. CBR was also proposed, which incorporates stakeholder preferences with requirement ordering, where estimates are computed using machine learning techniques. The primary objectives were to reduce the effort during the elicitation process and to make a partial order in attributes. The goal of the technique was to minimize the effort during the elicitation process [33]. Likewise, an intelligent solution was introduced for the prioritization of collected requirements from the stakeholders, in which the collected requirements are first grouped through the machine learning technique. Furthermore, the ANN tool is used, and finally, the AHP is used for comparison. During preceding clustering, stakeholders are asked to bring requirements,

and based on the profiles of stakeholders and as per the suggestions given by experts through ANN, requirements are prioritized. The ANN-based technique is better for large-sized requirements, but it is a highly complex process and difficult to use [8].

Another algorithmic technique called “DRANK” was suggested in [69] to handle dependency issues of the FRs. Nonetheless, this method cannot assign priority based on how much a requirement is necessary for other requirements and what prerequisite requirements are necessary for a particular requirement. Therefore, applying DRANK to large requirements will not be easy [69]. Based on the discussion of results in Table 9, we now generate Table 10, through which we can easily decide the selection of various techniques under different scenarios depending on the size of requirements and types of requirements.

TABLE 9 | Analysis of requirement prioritization techniques.

Technique	Usability (simple/ moderate/ complex)	Algorithmic (yes/no)	Time consuming (yes/moderately/not)	Suitable for which size of requirements (small/ medium/large)	Suitable for type of requirements (functional/ nonfunctional/ business)
AHP	Simple	No	Yes	Small	All types of requirements
Attribute goal oriented	Simple	No	Moderate	Small	Business requirements
Benefit and cost	Simple	No	Yes	Small	Business and user
Binary search tree	Simple	Yes	Yes	Medium	Functional requirements
Binary tree	Complex	Yes	Yes	Small	Functional requirements
Binary priority list	Simple	No	Yes	Medium	Functional requirements
Cased-based ranking	Simple	Yes	Yes	Small	All kinds
CBPA	Simple	No	Moderate	Small	All kinds
Cost value ranking	Simple	No	Yes	Small	Business requirements
Cognitive-driven requirement prioritization	Simple	No	Moderate	Small	Not mentioned
Voting based	Simple	No	Yes	Small	All kinds
Hierarchy AHP	Simple	No	Yes	Medium	All types
Interactive GA	Moderate	Yes	Yes	Small	User requirements/ functional
Lanchester theory	Simple	Not	Not	Small	Business
Numerical assignment	Simple	Not	Not	Small	All kinds
Value oriented	Simple	Not	Not	Small	Business
Value based	Simple	Not	Not	Small	User requirements
Value-based intelligent requirement prioritization	Difficult	Not	Not	Large	Business + User requirements
Group recommended algorithms	Difficult	Not	Not	Small	Business + User
Hierarchical cumulative voting	Simple	Not	Yes	Medium	All kinds
Machine learning	Difficult	Yes	Not	Large	User requirements
QFD	Easy	No	No	Small	Nonfunctional and business
Situation transition based	Easy	Not	Yes	Small	All kind
ANN	Difficult	Yes	No	Large	User requirements
Contextual preference-based technique	Easy	No	Yes	Small	Nonfunctional requirements
DRANK algorithm	Difficult	Yes	Moderate	Medium	Functional
Fuzzy decision logic	Moderate	Yes	Yes	Medium	All types
SNIPR	Moderate	Not	Yes	Large	Functional requirements
Tensor	Difficult	Not	Yes	Small	All types of requirements

TABLE 10 | Selection of the best technique based on size and type of requirements.

Size of requirements	Type of requirements	Suitable technique
Small	Business	1. AHP
		2. Attribute goal oriented
		3. Benefit and cost
		4. Binary tree
		5. Case-based ranking
		6. CBPA
		7. Cost value ranking
		8. Voting-based Lanchester theory
		9. Value oriented
		10. Numerical assignment
		11. Group recommended algorithms
		12. QFD, situation transition based
		13. Tensor
		14. Value-based intelligent requirement prioritization
Small	Functional	1. AHP
		2. Case-based ranking
		3. CBPA
		4. Voting based
		5. Interactive GA
		6. Numerical assignment
		7. Value based
		8. Group recommended algorithms
		9. Situation transition based
		10. Tensor
		11. Drank algorithms
Small	Nonfunctional	1. AHP
		2. Case-based ranking
		3. CBPA, voting-based
		4. Numerical assignment
		5. QFD
		6. Situation transition based
		7. Contextual preference-based technique
		8. Tensor
Small to medium	Functional requirements	1. Binary search tree
		2. Binary priority list
		3. Hierarchy AHP
		4. Hierarchical cumulative voting
		5. DRANK algorithm
		6. Fuzzy decision logic
Large	Functional requirements	1. Value-based intelligent requirement prioritization
		2. Machine learning
		3. ANN
		4. SNIPR

The identified techniques are suitable for prioritizing small-, medium-, or large-sized requirements. Furthermore, the requirements can be of any type: BRs, FRs, and NFRs. As illustrated in Table 10, it is evident that many of the suggested techniques are for prioritizing FRs, but the majority of them are based on stakeholders' inputs for prioritizing the selected features, and very

TABLE 11 | Prioritization techniques based on internal structure.

Prioritization technique	Based on stakeholders' preference	Based on the developer and internal structure
AHP	Yes	No
Case-based ranking	Yes	No
Voting-based technique	Yes	No
Interactive GA	No	Yes
Numerical assignment	Yes	No
Value-based technique	Yes	No
Group recommended algorithms	Yes	No
Situation transition based	Yes	No
Tensor	Yes	No
Drank algorithm	No	Yes
Binary search tree	Yes	No
Binary priority list	Yes	No
Hierarchy AHP	Yes	No
Hierarchical cumulative voting	Yes	No
Fuzzy decision logic	Yes	No
Value-based intelligent requirement prioritization	Yes	No
Machine learning-based techniques	Yes	No
ANN	Yes	No
SNIPR	Yes	No

little work has been done for prioritizing FRs from the development point of view. Such a comparison of the suitability of techniques based on the internal structure is presented in Table 11.

As shown in Table 11, we can observe that only two techniques prioritize the FRs based on the internal structure, which means the importance is determined by the software developers rather than the clients.

5 | Discussions

Dealing with FRs and their prioritization is particularly challenging, especially when the number of requirements is very large, such as those found in commercial business-oriented

TABLE 12 | Prioritization techniques with their limitations.

Technique	Limitations
SNIPR	Although good for prioritizing large requirements, it is a time-consuming and difficult process. The technique depends on the user rather than the internal structure of requirements.
Tensor	Although a suitable technique for small requirements in collaboration with nonfunctional requirements, it is a very difficult process that involves too much mathematics.
Drank algorithm	Prioritize and limit to functional requirements based on client's preferences rather than internal structure and from a development point of view.
ANN	Suitable technique for prioritizing features and big user requirements through neural networks but too complex in use and unable to prioritize functional requirements from a development point of view
Contextual preference-based technique	Only limited to prioritizing nonfunctional requirements that are very low in amount
Situation transition based	Technique mostly concentrates on behavioral perspectives of humans, beliefs, and environmental perspectives rather than functional requirements and their dependencies.
QFD	Limited to small-sized nonfunctional and business requirements
Machine learning	Besides the advantage that it can prioritize large-sized functional requirements, still, it does not solve dependency issues of functional requirements and is unable to prioritize functional requirements from a development point of view.
Hierarchical cumulative voting	Manual-based process in which stakeholders assign priority by giving points to requirements. Stakeholders give points to requirements, maybe some requirements are assigned with too many points, which can affect the overall results.
Group recommended algorithms	Accuracy is better but suitable for small-sized requirements.
Value-based intelligent requirement prioritization	Difficult to use and prioritize user requirements about business value. Does not solve functional requirements dependency issues from the developer's point of view
Value-based	Suitable for small-sized requirements and does not solve functional requirements dependency issues from the developer's point of view. Incapability to hierarchically shape objectives or requirements, poor handling of prerequisites
Value oriented	Limited to small-sized and business requirements only
Numerical assignment	It arranges requirements in different groups of high, medium, and low priority but does not prioritize requirements inside the group.
Lanchester theory	Do not describe comparative standards for the linguistic relations that can help the computation of comparative weights through all the requirements. Limited to prioritize business requirements only.
Interactive GA	Although manually related functional requirements and assigned priority but did not apply any standard algorithm. Further, the technique is not experimented on in real projects.
AHP	Manual assigning or comparative values to requirements rather than based on their internal structure of functional requirements. Too much time-consuming process

(Continues)

TABLE 12 | (Continued)

Technique	Limitations
Attribute goal oriented	Incompetent handling of the complications of the goal graphs. Limited technique to business and small-sized requirements.
Benefit and cost	Limited technique to business and user small-sized requirements. Do not accommodate for advancement of requirements; do not also weigh requirements based on a scale.
Binary search tree	Make only a simple classification of requirements without assigning any priority values. Does not solve dependency issues from the developer's point of view.
Binary tree	Complex and difficult to use. Does not solve dependency issues of functional requirements from a development point of view
Binary priority list	Does not solve dependency issues, and requirements are prioritized only based on their benefits rather than cost and other risks.
Cased-based ranking	Limited to small-sized requirements. Not able to bring collaboration among diverse stakeholders through cooperation and discussion
CBPA	Limited to small-sized requirements. Not scalable for functional requirements
Cost value ranking	Not so scalable. Limited for business requirements
Cognitive-driven requirement prioritization	This method does not include the procedure of combining and globalizing preference scores in the ranking process.
Voting-based technique	Limited for small-sized requirements and when stakeholders are not too much. As the number of stakeholders increases, issues arise. Better for one stakeholder. But still a manual process and based on stakeholder judgments instead of its internal structure and other dependencies on a business level.
Hierarchy AHP	Manual assigning or comparative values to requirements rather than based on their internal structure of functional requirements. Not suitable for big projects

ERP systems. Although several techniques were presented to prioritize a large set of requirements, most of them rely on the request of stakeholders or experts and are not based on the functional dependencies between one requirement and another. In addition to FRs, there is also a need to prioritize business PR as well as NFRs in complex software projects. The literature also indicated a lack of industrial evaluation necessitating the testing of proposed RP techniques in real-world domains. Furthermore, there is a potential opportunity for mobile software RP techniques as mobile software is becoming a trend in the market. Therefore, this area requires further investigation. Current prioritization methods are also prone to errors, as the guidelines prevailing in the RP practices in the present procedures are not sufficiently robust. This has also led to untrustworthy and unpredictable prioritization outcomes because such consequences do not reflect the accurate classification of requirements from the stakeholders' point of view or examination after the classification process. Therefore, vigorous algorithms are compulsory to produce reliable prioritization outputs. This research also observed that mostly the current prioritization techniques have not been applied in real-life developments, perhaps due to difficulties and problems related to prioritizations and the time necessary for producing prioritized requirements. Therefore, it is essential to practice the suggested frameworks and algorithms

that will progress or support RP at the commercial or industrial level. Table 12 summarizes all the identified techniques with limitations.

Specifically, it has been observed that publication biasness and incorrect data extraction have been identified as the major threats against this review protocol. All the papers were identified and collected using the searching strategy described in the protocol to answer the research questions. Still, there emerge the opportunities to miss important studies as not all the research can be searched with the help of the search terms associated with the research questions. To overcome the aforementioned shortcoming, going back to the references, all works and publications that have not been found using the design strategy were retrieved. Next, manual scanning and screening of all the identified papers and their references were conducted with much more patience to capture all the papers that might have been missed during the initial search. Additionally, a precise definition of the assortment procedure that meets the terms with the research questions was used to avert reject good studies inappropriately. Another issue is that some important papers were not included since they were written in a language other than English, and since our study only accepts papers in the English language, they were discarded.

6 | Conclusions

The purpose of this empirical study was to determine the scalability of current prioritization techniques to address the issue of software requirements and to explore their limitations and shortcomings. To achieve these goals, some of the research questions were refined based on an SLR. This was done by searching, assessing, and interpreting key and related articles. This research intended to identify areas of possible improvement through the methodical review of prior and related work on RP techniques as outlined in the literature. For this particular research study, a review protocol was developed and agreed upon to guide the research. As part of this research, some assessment questions or standards of quality were created and utilized in the existing work to determine its significance and perfection. This review protocol outlines each step of the review process that includes research identification, study selection, assessment of study quality, data extraction, and data management. These steps were performed according to the review protocol. During these changes, trends and related major studies were conducted, and the quality of these studies was assessed. After selecting various study types, information was then extracted from suitable papers. As a result, the research aim of the study was achieved, and the questions addressed in the paper were answered. This research successfully identified studies that addressed previous limitations and proposed improvements in prioritization techniques. Although a variety of prioritization methods are available, our findings suggest that further progress is still needed.

Key areas for enhancement include scalability, rigorous calculations, ease of use, consistency, the degree of trusting the result, industrial acceptance of the results, and conflicts of requirements.

Data Availability Statement

Data sharing not applicable to this article as no datasets were generated or analysed during the current study.

References

1. M. Yaseen, S. Ali, Abdullah, and N. Ullah, "An Improved Framework for Requirement Implementation in the Context of Global Software Development: A Systematic Literature Review Protocol," *International Journal of Database Theory and Application* 9, no. 6 (2016): 161–170.
2. B. D. Brock, "Towards Pattern-Driven Requirements Engineering: Development Patterns for Functional Requirements," in *8th International Model-Driven Requirements Engineering Workshop* (2018).
3. F. Khalique, W. Haider, and S. Ahmad, "Creating Domain Non-Functional Requirements Software Product Line Engineering Using Model Transformations," in *2017 International Conference on Frontiers of Information Technology (FIT)* (2017), 41–45.
4. X. Frank, Y. Sun, and C. Sekhar, "Priority Assessment of Software Process Requirements From Multiple Perspectives," *Journal of Systems and Software* 79 (2006): 1649–1660.
5. Guideline for the Selection of Requirement Prioritization Techniques in Agile Software Development: An Empirical Research," *International Journal of Recent Technology and Engineering* 8, no. 5 (2020): 3381–3388, <https://doi.org/10.35940/ijrte.e6634.018520>.

6. F. Hujainah, R. B. A. Bakar, A. B. Nasser, B. Al-haimi, and K. Z. Zamli, "SRPTackle: A Semi-Automated Requirements Prioritisation Technique for Scalable Requirements of Software System Projects," *Information and Software Technology* 131 (2021): 106501, <https://doi.org/10.1016/j.infsof.2020.106501>.
7. M. Yaseen, S. Baseer and S. Sherin, "Critical Challenges for Requirement Implementation in Context of Global Software Development, A Systematic Literature Review," in *International Conference on Open Source Systems and Technologies (ICOSST)* (2015), 120–125.
8. J. Erazo, H. Arboleda, and F. J. Pino, "Analysis of the Software Implementation Process for ERP Systems," *Springer International Publishing* 1 (2017): 297–312.
9. S. Parthasarathy and M. Daneva, "An Approach to the Estimation of the Degree of Customization for ERP Projects Using Prioritized Requirements," *Journal of Systems and Software* 117 (2016): 471–487.
10. A. S. Yaraghi, M. Bagherzadeh, N. Kahani, and L. Briand, "Scalable and Accurate Test Case Prioritization in Continuous Integration Contexts," *IEEE Transactions on Software Engineering* 49 (2022): 1–24, <https://doi.org/10.1109/tse.2022.3184842>.
11. M. A. Sami, Z. Rasheed, M. Waseem, Z. Zhang, T. Herda and P. Abrahamsson, "Prioritizing Software Requirements Using Large Language Models," arXiv.org, 2024, <https://arxiv.org/abs/2405.01564>.
12. E. Alhenawi, S. Awawdeh, R. A. Khurma, M. García-Arenas, P. A. Castillo and A. Hudaib, "Choosing a Suitable Requirement Prioritization Method: A Survey," arXiv.org, Feb. 20, 2024, <https://arxiv.org/abs/2402.13149>.
13. M. I. Babar, M. Ghazali, D. N. A. Jawawi, S. M. Shamsuddin, and N. Ibrahim, "PHandler: An Expert System for a Scalable Software Requirements Prioritization Process," *Knowledge-Based Systems* 84 (2015): 179–202.
14. R. Easmin, "A Partial Order Assimilation Approach for Software Requirements Prioritization," in *International Conference on Informatics, Electronics & Vision (ICIEV)* (2014), 1–5.
15. L. Alawneh, "Requirements Prioritization Using Hierarchical Dependencies," *Advances in Intelligent Systems and Computing* 558 (2018): 459–464.
16. M. Daneva and A. Herrmann, "Requirements Prioritization Based on Benefit and Cost Prediction: A Method Classification Framework," in *34th Euromicro Conference Software Engineering and Advanced Applications* (2008), 240–247.
17. B. B. Jawale, G. K. Patnaik and A. T. Bhole, "Requirement Prioritization Using Adaptive Fuzzy Hierarchical Cumulative Voting," in *IEEE 7th International Advance Computing Conference (IACC)* (2017).
18. S. Dhingra and M. Madan, "Selection of Prioritization Technique for Software Requirement Using Fuzzy Logic and Decision Tree," in *Online International Conference on Green Engineering and Technologies (IC-GET)* (2016).
19. R. Beg, R. P. Verma and A. Joshi, "Reduction in Number of Comparisons for Requirement Prioritization Using B-Tree," in *2009 IEEE International Advance Computing Conference*, no. March (2009), 6–7.
20. A. Perini, A. Susi, and P. Avesani, "A Machine Learning Approach to Software Requirements Prioritization," *IEEE Transactions on Software Engineering* 39, no. 4 (2013): 445–461.
21. R. K. Chopra, V. Gupta, and D. S. Chauhan, "Experimentation on Accuracy of Non Functional Requirement Prioritization Projects," *Perspectives on Science* 8 (2016): 79–82.
22. N. Setiani and T. Dirgahayu, "Clustering Technique for Information Requirement Prioritization in Specific CMSs," in *International Conference on Data and Software Engineering (ICoDSE)* (2016).
23. J. C. B. Somohano-Murrieta, J. O. Ocharan-Hernandez, A. J. Sanchez-Garcia, and M. de los Angeles Arenas-Valdes, "Requirements

- Prioritization Techniques in the Last Decade: A Systematic Literature Review," in *2020 8th International Conference in Software Engineering Research and Innovation (CONISOFT)* (2020), <https://doi.org/10.1109/conisof50191.2020.00013>.
24. R. Anwar and M. B. Bashir, "A Systematic Literature Review of AI-Based Software Requirements Prioritization Technique," *IEEE Access* 11 (2023): 143815–143860, <https://doi.org/10.1109/access.2023.3343252>.
 25. F. Hujainah, R. B. A. Bakar, M. A. Abdulgaber, and K. Z. Zamli, "Software Requirements Prioritisation: A Systematic Literature Review on Significance, Stakeholders, Techniques and Challenges," *IEEE Access* 6 (2018): 71497–71523, <https://doi.org/10.1109/access.2018.2881755>.
 26. Muhammad Sufian, Z. Khan, S. Rehman, and Wasi Haider Butt, "A Systematic Literature Review: Software Requirements Prioritization Techniques," (2018), <https://doi.org/10.1109/fit.2018.00014>.
 27. F. A. Bukhsh, Z. A. Bukhsh, and M. Daneva, "A Systematic Literature Review on Requirement Prioritization Techniques and Their Empirical Evaluation," *Computer Standards & Interfaces* 69 (2020): 103389, <https://doi.org/10.1016/j.csi.2019.103389>.
 28. N. Riegel and J. Doerr, "A Systematic Literature Review of Requirements Prioritization Criteria," in *International Working Conference on Requirements Engineering: Foundation for Software Quality* (2015), 300–317, https://doi.org/10.1007/978-3-319-16101-3_22.
 29. P. Achimugu, A. Selamat, R. Ibrahim, and M. N. Mahrin, "A Systematic Literature Review of Software Requirements Prioritization Research," *Information and Software Technology* 56, no. 6 (2014): 568–585, <https://doi.org/10.1016/j.infsof.2014.02.001>.
 30. R. Beg, Q. Abbas, and R. P. Verma, "An Approach for Requirement Prioritization using BTree," *First International Conference on Emerging Trends in Engineering and Technology* 7 (2008): 1216–1221.
 31. S. U. Khan, M. Niazi, and R. Ahmad, "Factors Influencing Clients in the Selection of Offshore Software Outsourcing Vendors: An Exploratory Study Using a Systematic Literature Review," *Journal of Systems and Software* 84, no. 4 (2011): 686–699.
 32. M. Niazi, et al., "Challenges of Project Management in Global Software Development: A Client-Vendor Analysis," *Information and Software Technology* 80 (2016): 1–19.
 33. A. Nguyen-duc, D. S. Cruzes, and R. Conradi, "The Impact of Global Dispersion on Coordination, Team Performance and Software Quality—A Systematic Literature Review," *Information and Software Technology* 57 (2015): 277–294.
 34. M. Yaseen, A. Mustapha, M. A. Shah, and N. Ibrahim, "A Hybrid Technique Using Minimal Spanning Tree and Analytic Hierarchical Process to Prioritize Functional Requirements for Parallel Software Development," *Requirement Engineering Journal* 28 (2023): 347–376.
 35. J. M. Verner, O. P. Brereton, B. A. Kitchenham, M. Turner, and M. Niazi, "Risks and Risk Mitigation in Global Software Development: A Tertiary Study," *Information and Software Technology* 56, no. 1 (2014): 54–78.
 36. M. A. Iqbal, A. M. Zaidi and S. Murtaza, "A New Requirement Prioritization Model for Market Driven Products Using Analytical Hierarchical Process," in *DSDE 2010 - Int. Conf. Data Storage Data Eng.*, 2010, 142–149.
 37. N. Garg, M. Sadiq and P. Agarwal, "GOASREP: Goal Oriented Approach for Software Requirements Elicitation and Prioritization Using Analytic Hierarchy Process," in *Proceedings of the 5th International Conference on Frontiers in Intelligent Computing: Theory and Applications* (2017), 281–287.
 38. E. Bagheri, M. Asadi, D. Gasevic, and S. Soltani, "Stratified Analytic Hierarchy Process: Prioritization and Selection of Software Features," *International Conference on Software Product Lines* 6287 (2010): 300–315.
 39. A. Perini, F. Ricca, and A. Susi, "Tool-Supported Requirements Prioritization: Comparing the AHP and CBRank Methods," *Information and Software Technology* 51, no. 6 (2009): 1021–1032.
 40. A. Perini, F. Ricca, F. Bruno, K. Irst and A. Susi, "An Empirical Study to Compare the Accuracy of AHP and CBRanking Techniques for Requirements Prioritization," in *Fifth International Workshop on Comparative Evaluation in Requirements Engineering* (2007).
 41. M. Dabbagh and S. P. Lee, "A Consistent Approach for Prioritizing System Quality Attributes," in *SNPD 2013-14th ACIS Int. Conf. Softw. Eng. Artif. Intell. Netw. Parallel/Distributed Comput.* (2013), 317–322.
 42. S. I. Mohamed, I. A. M. El-maddah, and A. M. Wahba, "Towards Value-Based Requirements Prioritization for Software Product Management," *International Journal of Software Engineering* 1 (2008): 1–14.
 43. N. Garg, "Recent Advancements in Requirement Elicitation and Prioritization Techniques," *International Conference on Advances in Computer Engineering and Applications* 4 (2015): 237–240.
 44. M. Aasem, M. Ramzan, A. Jaffar and E. S. Islamabad, "Analysis and Optimization of Software Requirements Prioritization Techniques," in *International Conference on Information and Emerging Technologies* (2010).
 45. A. Herrmann and M. Daneva, "Requirements Prioritization Based on Benefit and Cost Prediction: An Agenda for Future Research," in *16th IEEE International Requirements Engineering Conference*, no. iv (2008), 125–134.
 46. A. Ahmad, A. Shahzad, V. K. Padmanabhuni, A. Mansoor, S. Joseph and Z. Arshad, "Requirements Prioritization With Respect to Geographically Distributed Stakeholders," in *IEEE International Conference on Computer Science and Automation Engineering* (2011).
 47. T. Bebensee, I. Van De Weerd, and S. Brinkkemper, "Binary Priority List for Prioritizing Software Requirements," in *International Working Conference on Requirements Engineering: Foundation for Software Quality* (2010), 67–78.
 48. J. Karlsson, "A Cost-Value Approach for Prioritizing Requirements," *IEEE Software* 14 (1997): 67–74.
 49. P. Chatzipetrou, L. Angelis, P. Rovegård and C. Wohlin, "Prioritization of Issues and Requirements by Cumulative Voting: A Compositional Data Analysis Framework," in *Proceedings. - 36th EUROMICRO Conf. Softw. Eng. Adv. Appl. SEAA 2010* (2010), 361–370.
 50. M. Svahnberg, "A Study on the Importance of Order in Requirements Prioritisation," in *Third International Workshop on Software Product Management* (2010).
 51. R. M. Liaqat, "A Majority Voting Goal Based Technique for Requirement Prioritization," in *22nd IEEE International Conference on Automation and Computing (ICAC)* (2016).
 52. M. A. Awais, "Requirements Prioritization: Challenges and Techniques for Quality Software Development," *ACSII Advances in Computer Science: An International Journal* 5, no. 2 (2016): 14–21.
 53. P. Tonella, A. Susi, and F. Palma, "Interactive Requirements Prioritization Using a Genetic Algorithm," *Information and Software Technology* 55, no. 1 (2013): 173–187.
 54. T. M. Fehlmann, "New Lanchester Theory for Requirements Prioritization," *Second International Workshop on Software Product Management* 8 (2008): 1–6.
 55. A. K. Massey, P. N. Otto, and A. I. Antón, "Prioritizing Legal Requirements," *Second International Workshop on Requirements Engineering and Law* 111 (1936): 2010.
 56. P. Voola and V. Babu, "Study of Aggregation Algorithms for Aggregating Imprecise Software Requirements' Priorities," *European Journal of Operational Research* 259, no. 3 (2017): 1191–1199.

57. M. A. A. Elsood and H. A. Hefny, "A Goal-Based Technique for Requirements Prioritization," in *9th International Conference on Informatics and Systems (INFOS2014)* (2014).
58. N. Misaghian and H. Motameni, "An Approach for Requirements Prioritization Based on Tensor Decomposition," *Requirements Engineering* 23, no. 2 (2018): 169–188.
59. A. S. Danesh, "Requirements Prioritization in On-Line Banking Systems: Using Value-Oriented Framework," *International Conference on Computer Technology and Development* 9 (2009): 158–161.
60. N. Mohamed, S. Mazen, and W. Helmy, "E-AHP: An Enhanced Analytical Hierarchy Process Algorithm for Prioritizing Large Software Requirements Numbers," *International Journal of Advanced Computer Science and Applications* 13, no. 7 (2022): 190–199, <https://doi.org/10.14569/ijacsa.2022.0130725>.
61. T. Amelia and R. Mohamed, "A Review: Requirements Prioritization Criteria Within Collaboration Perspective," *Pertanika Journal of Science and Technology* 31, no. 1 (2022): 161–185, <https://doi.org/10.47836/pjst.31.1.11>.
62. S. S. Payyavula and F. Si, "Application of Value Based Requirement Prioritization in a Banking Product Implementation," *Third International Conference on Services in Emerging Markets* 12 (2012): 157–161.
63. J. Mczara, S. Sarkani, T. Holzer, and T. Eveleigh, "Software Requirements Prioritization and Selection Using Linguistic Tools and Constraint Solvers—A Controlled Experiment," *Empirical Software Engineering* 20, no. 6 (2015): 1721–1761.
64. N. Kukreja, S. Swaroop, B. Boehm, and S. Padmanabhuni, "Value-Based Requirements Prioritization: Usage Experiences," *Procedia Computer Science* 16 (2013): 806–813.
65. M. Ramzan, M. A. Jaffar, and A. A. Shahid, "Value Based Intelligent Requirement Prioritization (Virp): Expert Driven Fuzzy Logic Based Prioritization Technique," *International Journal of Innovative Computing, Information and Control* 7, no. 3 (2011): 1017–1038.
66. M. Ramzan and M. A. Jaffar, "Value Based Fuzzy Requirement Prioritization and Its Evaluation Framework," in *Fourth International Conference on Innovative Computing, Information and Control (ICICIC)* (2009), 1464–1468.
67. A. Felfernig and G. Ninaus, "Group Recommendation Algorithms for Requirements Prioritization," in *Third International Workshop on Recommendation Systems for Software Engineering (RSSE)* (2012), 59–62.
68. R. Santos, A. Albuquerque, and P. R. Pinheiro, "Towards the Applied Hybrid Model in Requirements Prioritization," *Procedia Computer Science* 91 (2016): 909–918.
69. C. Duan and Æ. P. Laurent, "Towards Automated Requirements Prioritization and Triage," *Journal of Requirement Engineering* 14 (2009): 73–89.
70. I. Eitiveni and H. A. Hutapea, "Prioritizing Critical Success Factors of Requirements Engineering Using Analytical Hierarchy Process," *Jurnal Sistem Informatika* 19, no. 1 (2023): 13–27, <https://doi.org/10.21609/jsi.v19i1.1197>.
71. A. Salleh, M. Y. Said, M. H. Osman, and S. Hassan, "A Review on Classifying and Prioritizing User Review-Based Software Requirements," *JOIV: International Journal on Informatics Visualization* 8, no. 3–2 (2024): 1651, <https://doi.org/10.62527/joiv.8.3-2.3450>.
72. P. Taylor, H. Raharjo, M. Xie, and A. C. Brombacher, "Prioritizing Quality Characteristics in Dynamic Quality Function Deployment," *International Journal of Production Research* 44, no. 23 (2006): 5005–5018.
73. C. E. Otero, E. Dell, A. Qureshi and L. D. Otero, "A Quality-Based Requirement Prioritization Framework Using Binary Inputs," in *Fourth Asia International Conference on Mathematical/Analytical Modelling and Computer Simulation* (2010), 0–5.
74. N. L. Atukorala, C. K. Chang, and K. Oyama, "Situation-Oriented Evaluation and Prioritization of Requirements," *Asia Pacific Requirements Engineering Conference* 1 (2016): 18–33.
75. S. Khanneh and V. Anu, "Security Requirements Prioritization Techniques: A Survey and Classification Framework," *Software* 1, no. 4 (2022): 450–472, <https://doi.org/10.3390/software1040019>.
76. E. Serral, P. Senany, A. F. Dragoni, and F. Dalpiaz, "Contextual Requirements Prioritization and Its Applications to Smart Homes," *Journal of Systems and Software DRank* 126 (2017): 141–156.
77. F. Shao, R. Peng, H. Lai, and B. Wang, "A Semi-Automated Requirements Prioritization Method Based on Preferences and Dependencies," *Journal of Systems and Software DRank* 126 (2017): 141–156.
78. F. Hassan, T. T. Nguyen, T. Le, and C. Le, "Automated Prioritization of Construction Project Requirements Using Machine Learning and Fuzzy Failure Mode and Effects Analysis (FMEA)," *Automation in Construction* 154 (2023): 105013–105013, <https://doi.org/10.1016/j.autcon.2023.105013>.